Проектирование архитектуры и конструирование ПО *

Проектирование ОС, Технологии, Инструменты

Лекция 1

Ознакомительная

Экосистема



На Рис. 1 представлена экосистема, состоящая из аппаратной части, которая не может существовать без программной части для выполнения задач, поставленных пользователем.



ГОСТы и Международные стандарты

- **Понятие «ЭВМ»**

Согласно ГОСТу 15971-90 «СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ термины и определения», для термина **ЭВМ** дано следующее определение:

ГОСТ 15971-90: Электронная вычислительная машина

Вычислительная машина, основные функциональные устройства которой выполнены на электронных компонентах.



ГОСТы и Международные стандарты

- **Понятие «Операционная система»**

ГОСТ 15971-90: Операционная Система

Совокупность системных программ, предназначенная для обеспечения определенного уровня эффективности системы обработки информации за счет автоматизированного управления ее работой и предоставляемого пользователю определенного набора услуг



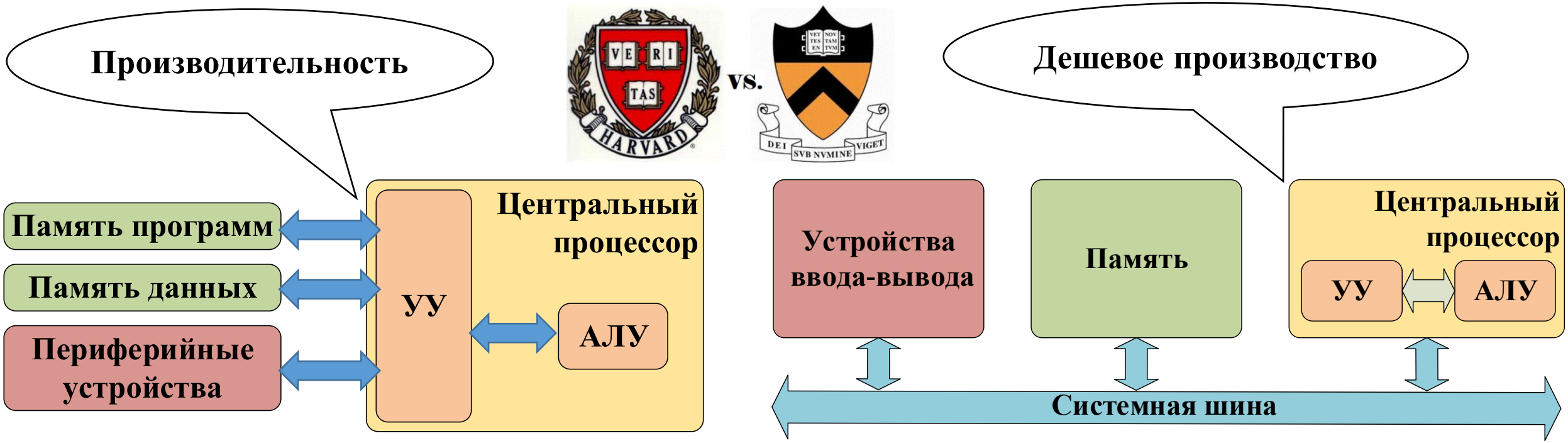
Аппаратная архитектура (Классификация)

- По типу применяемого процессора
  - CISC (англ. complex instruction set computing) — архитектура с полным набором команд.
  - RISC (англ. reduced instruction set computing) — архитектура с сокращённым набором команд.
  - MISC (англ. minimal instruction set computing) — архитектура с минимальным набором команд.
  - VLIW (англ. very long instruction word — «очень длинная машинная команда») — архитектура с длинной машинной командой, в которой указывается параллельность выполнения вычислений.

Характеристика	CISC	RISC	VLIW
Длина команды	Различная	Одинаковая	Одинаковая
Количество регистров	Небольшое. Регистры специализированные	Большое. Регистры универсальные.	Большое. Регистры универсальные.
Доступ к памяти	Кодируется в команде. Выполняется по микрокоманде	Выполняется по специальной команде	Выполняется по специальной команде
Длительность выполнения команд	Различная	Одинаковая (для большинства	Различная

Аппаратная архитектура (Классификация)

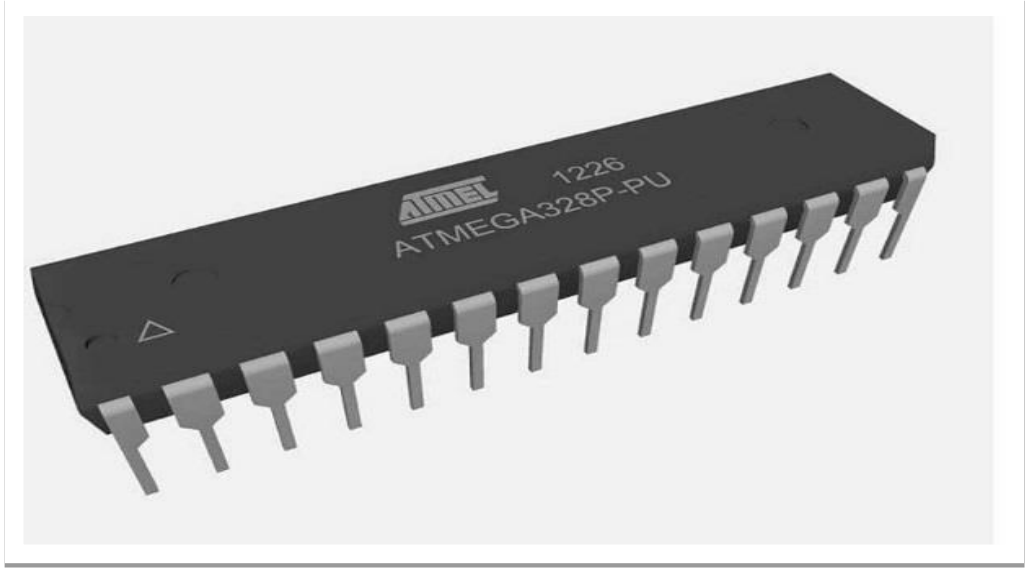
- По принципу разделения памяти
  - Гарвардская архитектура — характерной чертой является разделение памяти программ и памяти данных.
  - Фон Неймановская архитектура — характерной чертой является совместное хранение программ и данных.



Аппаратная архитектура (Классификация)

- По исполнению устройств:
  - Микропроцессор (CPU - Central Processing Unit)
  - Микроконтроллер (MCU - Micro Controller Unit)





Аппаратная архитектура (Классификация)

- По исполнению устройств:
  - Система на кристалле (SoC - System-on-a-Chip)
  - Программируемая логическая интегральная схема (PLD - Programmable Logic Device)







Программная архитектура (Классификация)

- По области применения:
  - ОС для мэйнфреймов (больших компьютеров)
  - ОС для серверов
  - ОС для персональных компьютеров
  - ОС для мобильных устройств

  - ***ОС для встраиваемых систем*** (бытовая электроника, промышленная автоматика, транспорт,телекоммуникационные системы, медицинское оборудование, аэрокосмическая техника, робототехника и т.п.)

  - ОС для коммуникационного оборудования (маршрутизаторы, терминалы, модемы)
  - ОС для ИИ



Программная архитектура (Классификация)

- По архитектуре процессоров (CPU, MCU, SoC):
  - x86
  - AVR
  - ARM
  - MIPS
  - POWER-PC
  - RISC-V
- По количеству процессоров:
  - Однопроцессорные
  - Многопроцессорные
- По разрядности кода:
  - 8-разрядная
  - 16-разрядная
  - 32-разрядная
  - 64-разрядна



Программная архитектура (Классификация)

- По количеству пользователей:
  - Однопользовательские ОС
  - Многопользовательские ОС
- По числу процессов:
  - Однозадачные
  - Многозадачные
- По характеру взаимодействия:
  - Встроенные
  - Пакетные
  - Интерактивные (с разделением времени)
  - Реального времени



Программная архитектура (Классификация)

- По типу ядра ОС:
  - Монолитное ядро
  - Модульное ядро
  - ***Микроядро***
  - Гибридное ядро
  - Экзоядро
  - ***Unikernel*** (зависит от ядра хоста)
- По типу лицензии:
  - Коммерческая лицензия
  - Условно-бесплатная лицензия
  - Бесплатная лицензия
  - Свободная лицензия

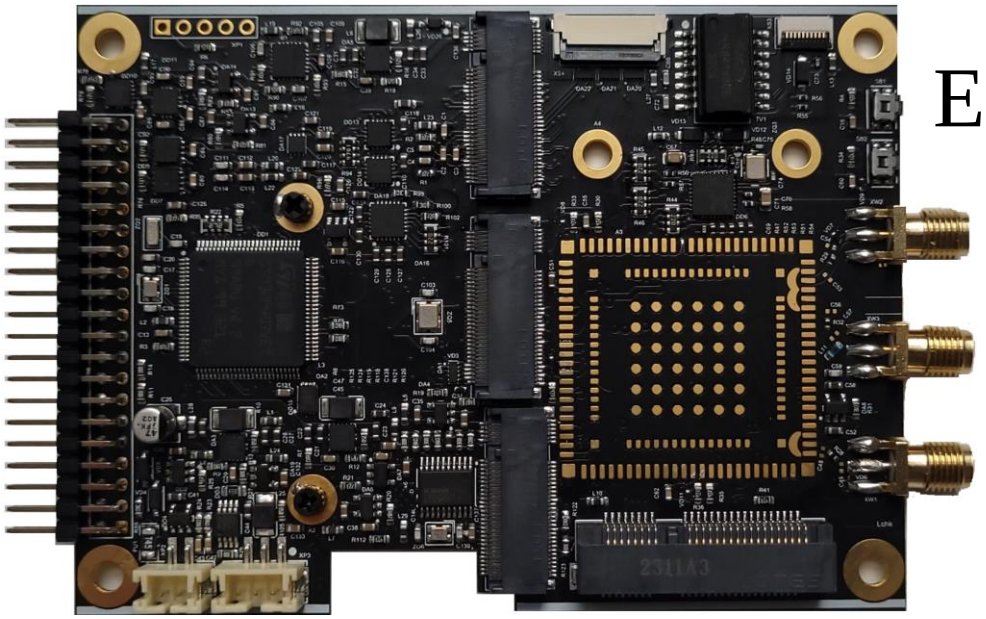


Что изучать?

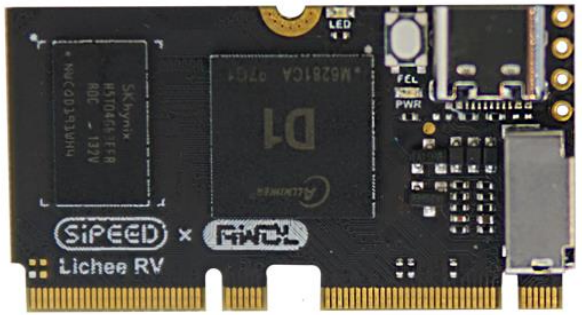
- Из классификации аппаратной и программной частей виден объем и сложность темы ОС.
- К сложности темы ОС добавляется проблематика комплекса задач, которые должна решать ОС.
- В технологическом развитии также видна необходимость в проектировании и разработке ОС нового поколения.
- При этом стоит учитывать и правовые аспекты.

Очевидно должна быть учебная модель, которая позволяла изучить концепцию организации, управления и разработки современных ОС

Аппаратная часть



Eco SB



SoM
Lichee RV





AI - M1n

- Разработан Пирф в Сколково (Designed by Peerf in Sk)
  - За исключением самого модуля Lichee RV (SiPeed)
- Собран в Китае (Assembled in China)
  - Комплектующие полностью импортные

- Но мы верим, что в России появятся заводы по производству процессоров, микроконтроллеров на RISC-V архитектуре, а также сопутствующих комплектующих



Программная часть

ЭКООС



- Полностью отечественная разработка компании Пирф НН.
- Запатентованная архитектура Эко ОС
- Программные компоненты разрабатываются на базе международных и открытых стандартов, а также внутренних технологических стандартах компании Пирф

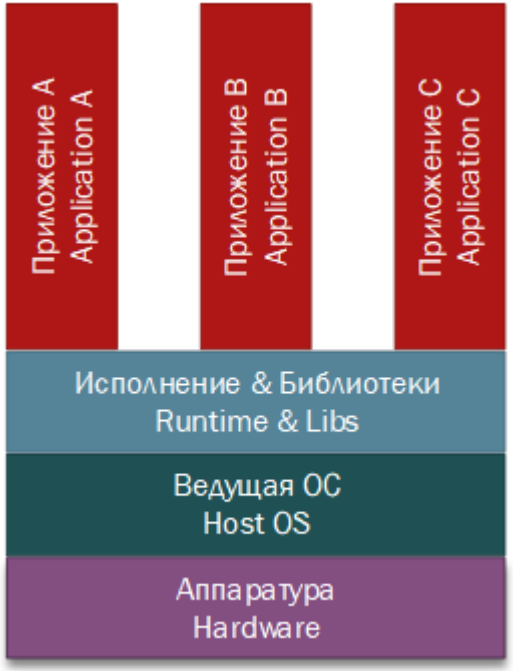


- Ни каких клонов существующих ОС, но отчасти вы можете создавать Linux, Windows, Mac OS X подобные ОС на базе компонентов компонентной среды Эко-платформы



Программная часть

- Позволяет создавать решения:
  - для встраиваемой систем используя микроядро Эко ОС (EcoOS.Microkernel) на голом железе;
  - для платформ Windows, Linux, MacOSX, iOS, Android используя унификационное ядро Эко ОС (EcoOS.Unikernel)



Bare metal



Unikernel

Программная архитектура (МикроЯдро ОС)



РОССИЙСКАЯ ФЕДЕРАЦИЯ

ПАТЕНТ

НА ИЗОБРЕТЕНИЕ

№ 2718235

АРХИТЕКТУРА ОПЕРАЦИОННОЙ СИСТЕМЫ ДЛЯ ОБЕСПЕЧЕНИЯ ПОДДЕРЖКИ ПОКОЛЕНИЙ МИКРОЯДЕР

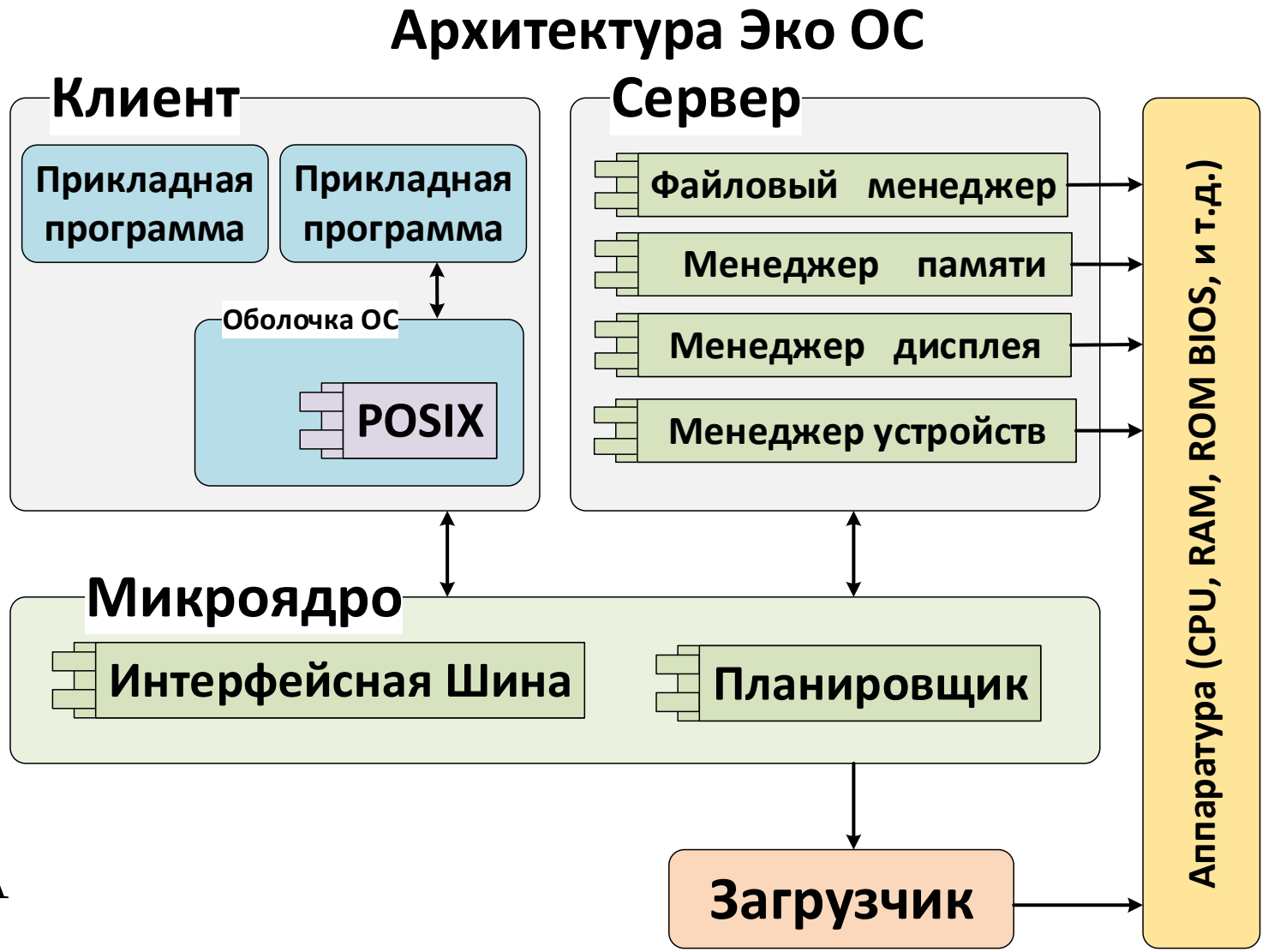
Патентообладатель: *Общество с ограниченной ответственностью «ПИРФ» (ООО «ПИРФ») (RU)*

Авторы: *Ильин Николай Олегович (RU), Башев Владимир Николаевич (RU)*

Заявка № 2019119349
Приоритет изобретения 21 июня 2019 г.
Дата государственной регистрации в Государственном реестре изобретений Российской Федерации 31 марта 2020 г.
Срок действия исключительного права на изобретение истекает 21 июня 2039 г.

Руководитель Федеральной службы по интеллектуальной собственности Г.П. Ивлиев

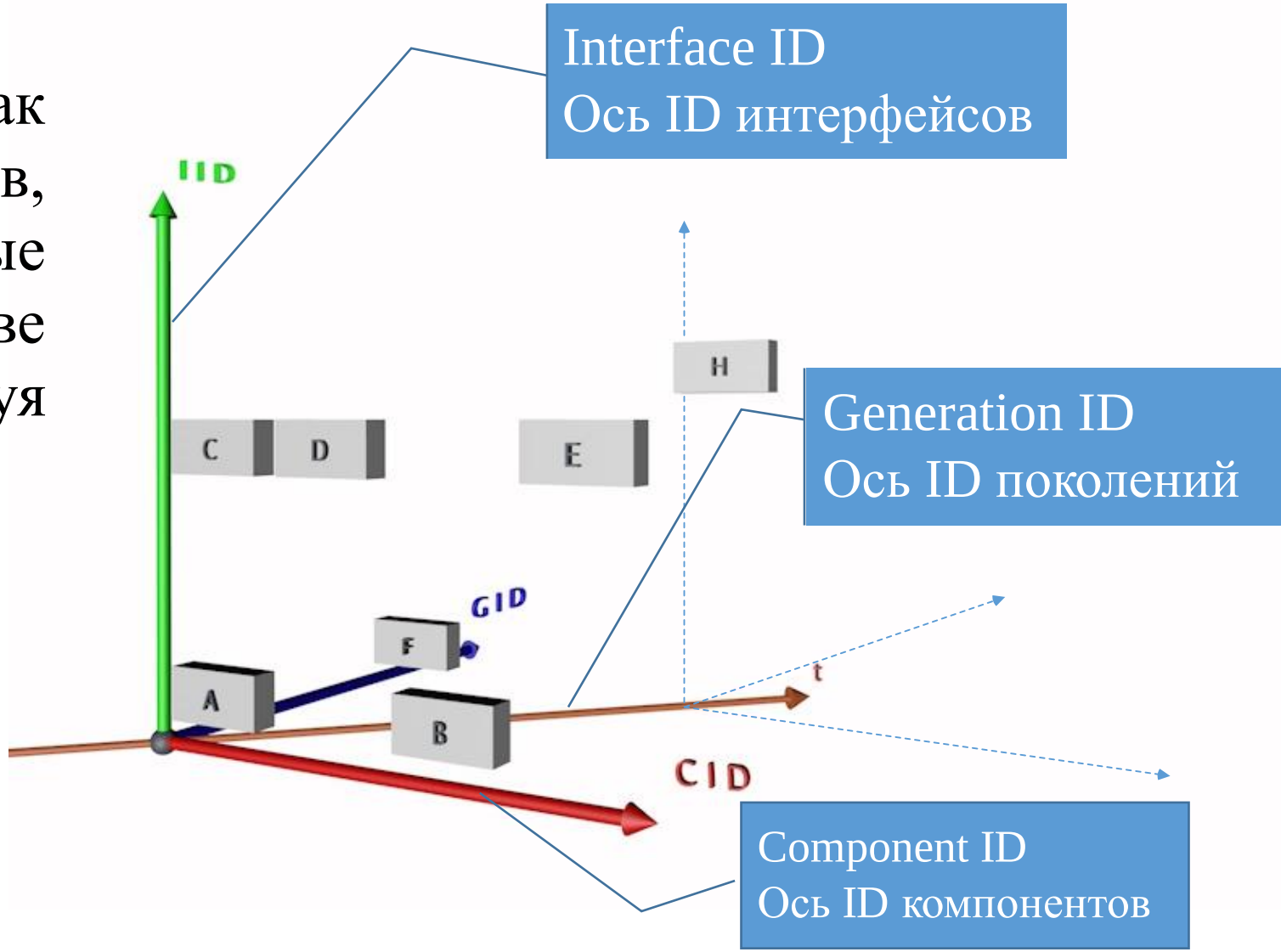
Россия № 2718235
США № US11520597B2
Китай №CN202010562047.2A
ЕС № EP20187690.1A



Технология:
Адаптированная Объектная Модель Компонентов (АОМК)

АОМК - технология, как конструктор LEGO из блоков, позволяет создавать различные решения, только в качестве строительных блоков используя компоненты.

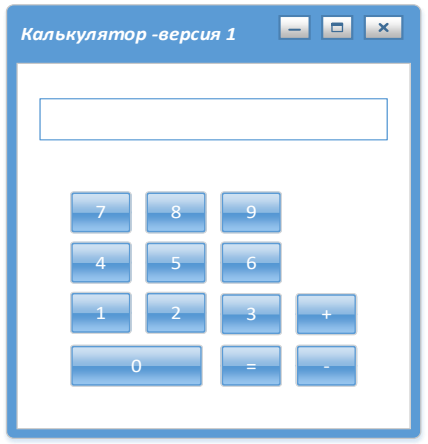


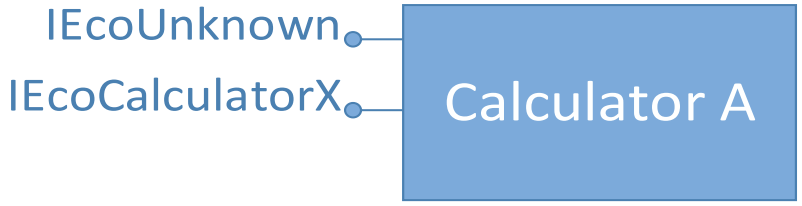


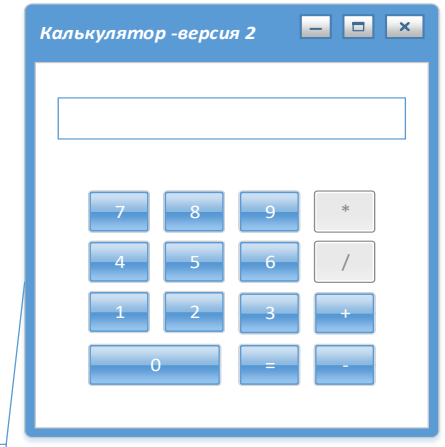


Технология:

Пример: варианты использования

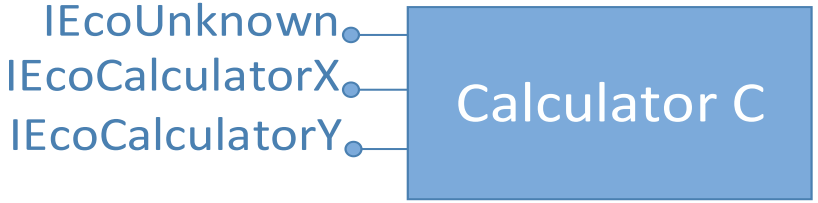


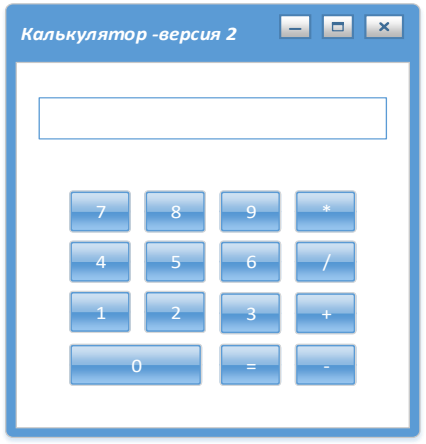




Калькулятор в. 2 использует интерфейс IEcoCalculatorX, Доступны только две функции сложение и вычитание







Калькулятор в. 1 использует интерфейс IEcoCalculatorX и ничего не знает о интерфейсе IEcoCalculatorY



Инструменты

- На сегодняшний день невозможно представить разработку ПО без набора инструментов (toolchain) :
  - Редакторы кода
  - Компиляторы
  - Компоновщики
  - Отладчики
  - Вспомогательные инструменты для поддержки производственного цикла ПО



ГОСТы и Международные стандарты

- **Понятие «Компилятор»**

ГОСТ 15971-90: Компилятор

Программа или техническое средство, выполняющее компиляцию

- **Понятие «Компиляция»**

ГОСТ 15971-90: Компиляция

Трансляция программы с языка высокого уровня в форму, близкую к программе, на машинном языке

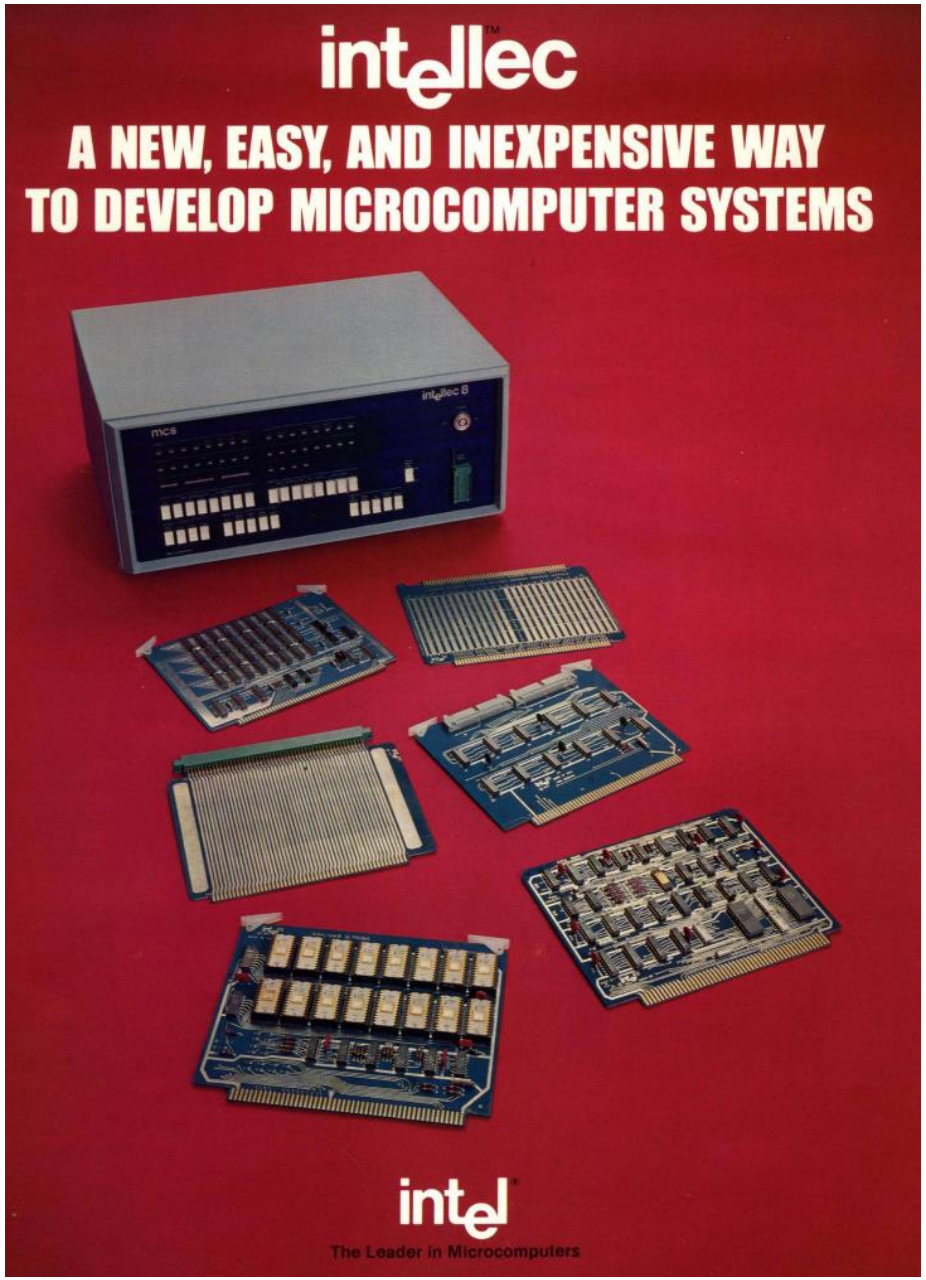
С чего начать?

- **Задача:** Необходимо разработать ОС для новой архитектуры процессора (для первого ПК)



Где взять инструменты?

- Для первых ПК (IBM PC), точнее новой архитектуры процессора такой как Intel 4004 - 4 bit и 8008 - 8 bit (1972 год), а позднее для 16 - разрядной архитектуры Intel 8086 (1976 - 1978 г.), компания Intel предоставляла систему разработки микрокомпьютеров под названием Intellect (Интеллект).



Инструменты: техническое средство

Intel Intellect 4 :
Программирование + программатор ПЗУ





Intel - Intellect 8

Intellec включал в себя:

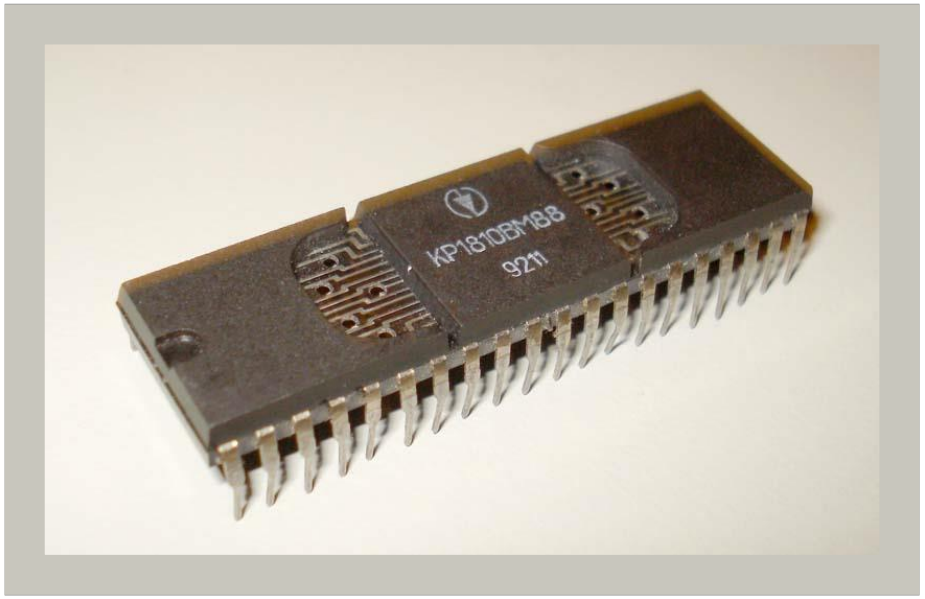
- ассемблер;
- компоновщик;
- отладчик;
- внутрисхемный эмулятор

Характеристики:

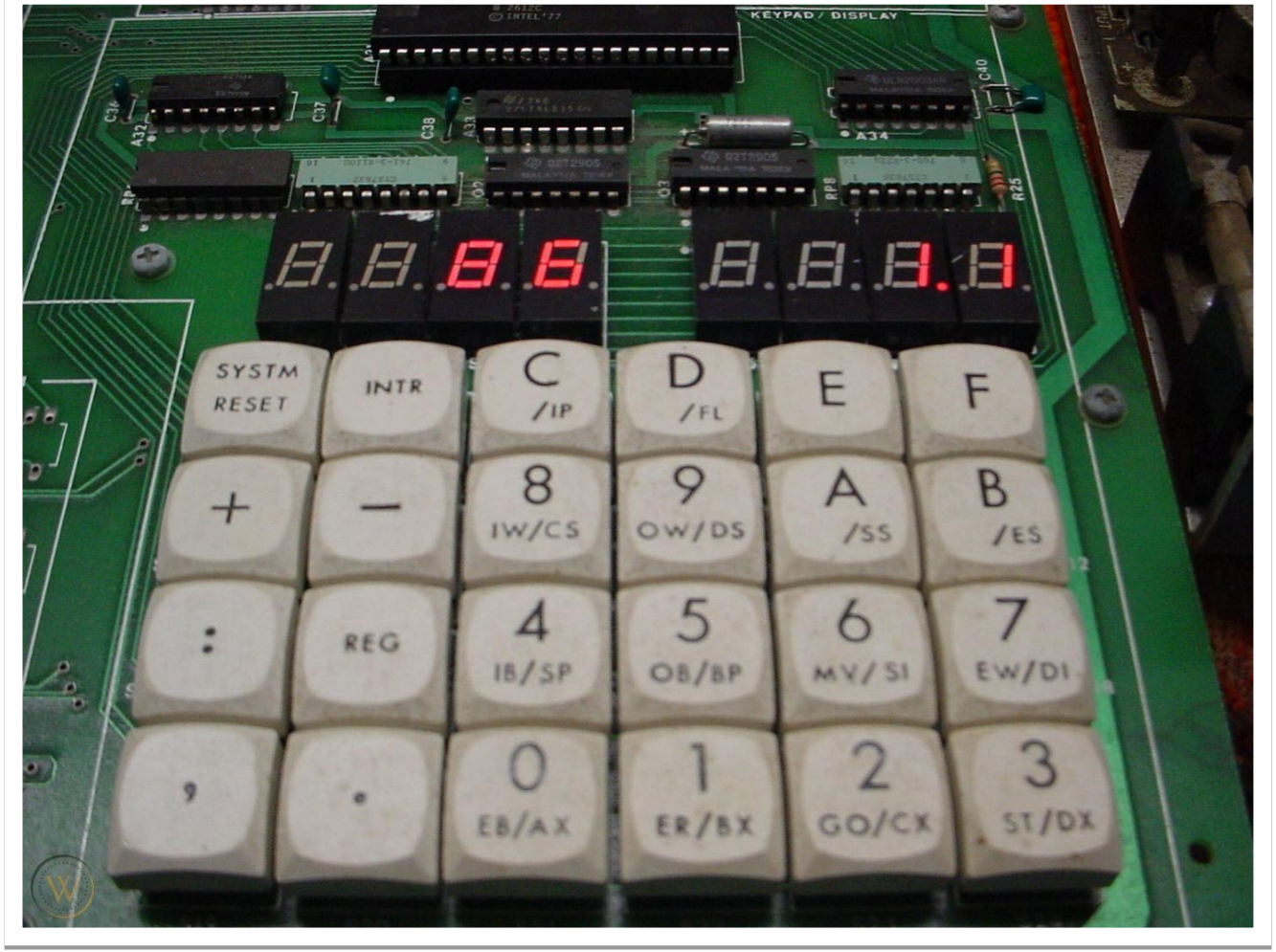
- способен адресовать до 16 КБ;
- работал с частотой 800 кГц;
- время цикла команды 12,5 мкс



Отечественный аналог К1810ВМ86 (I8086)



- это советский 16-разрядный микропроцессор, клон процессора Intel 8086, с которым он совместим в двоичном формате и с выводами





Популярная альтернатива для I8080 в СССР/России

- Это Z80 (Т34ВМ1)
  - Программировать можно на BASIC и ASSEMBLER
  - Для ввода используется интерпретатор BASIC

Пример:

10 PRINT “HELLO”

>RUN



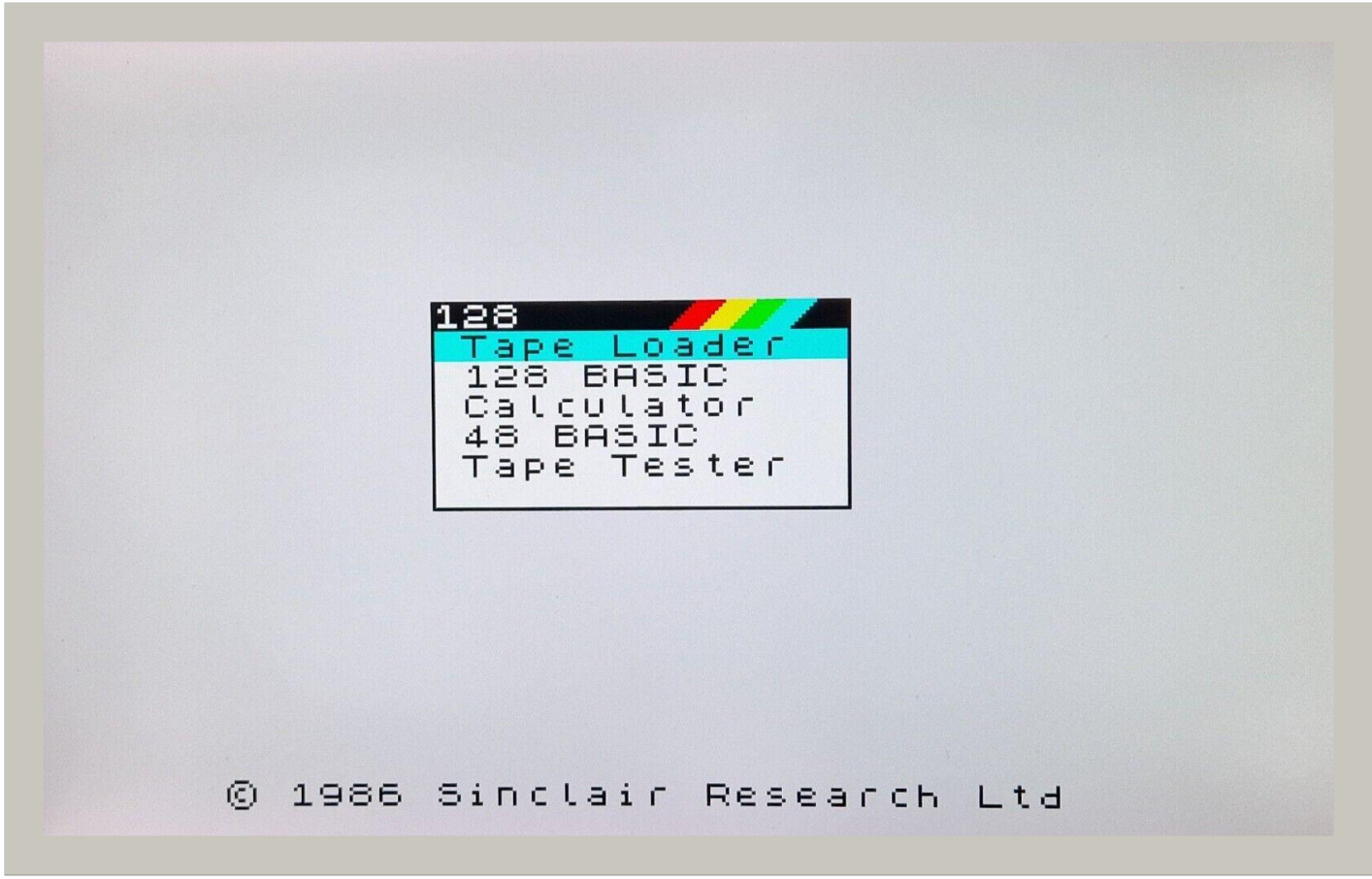


Пример программирования

- На ассемблере:

```
10 REM LD A, “H”
20 POKE 50000,62: POKE 50001,72
30 REM RST 16
40 POKE 50002,215
50 REM LD A, “E”
60 POKE 50003,62: POKE 50004,69
70 REM RST 16
80 . . .
```

>RANDOMIZE USR 5000



Начальный экран запуска ПК



Пример программирования

- На бейсике:

```
10 LET xmax = 10: LET ymax = 10
20 LET xmin =10: LET ymin = 10
30 LET xh = (xmaxxmin)/23: LET yh =
    (ymaxymin)/23
40 LET scale = 255/SQR(3)*2/((xmaxxmin) *
    (ymaxymin))
50 LET zscale = 40
60 DIM c(256): FOR i = 1 TO 256: LET c(i)=0: NEXT
70 FOR y=ymax TO ymin STEP yh: LET y1 = y*scale
80 FOR x=xmax TO xmin STEP xh: LET x1=x*scale
90 GO SUB 5000
100 . . .

>RUN
```

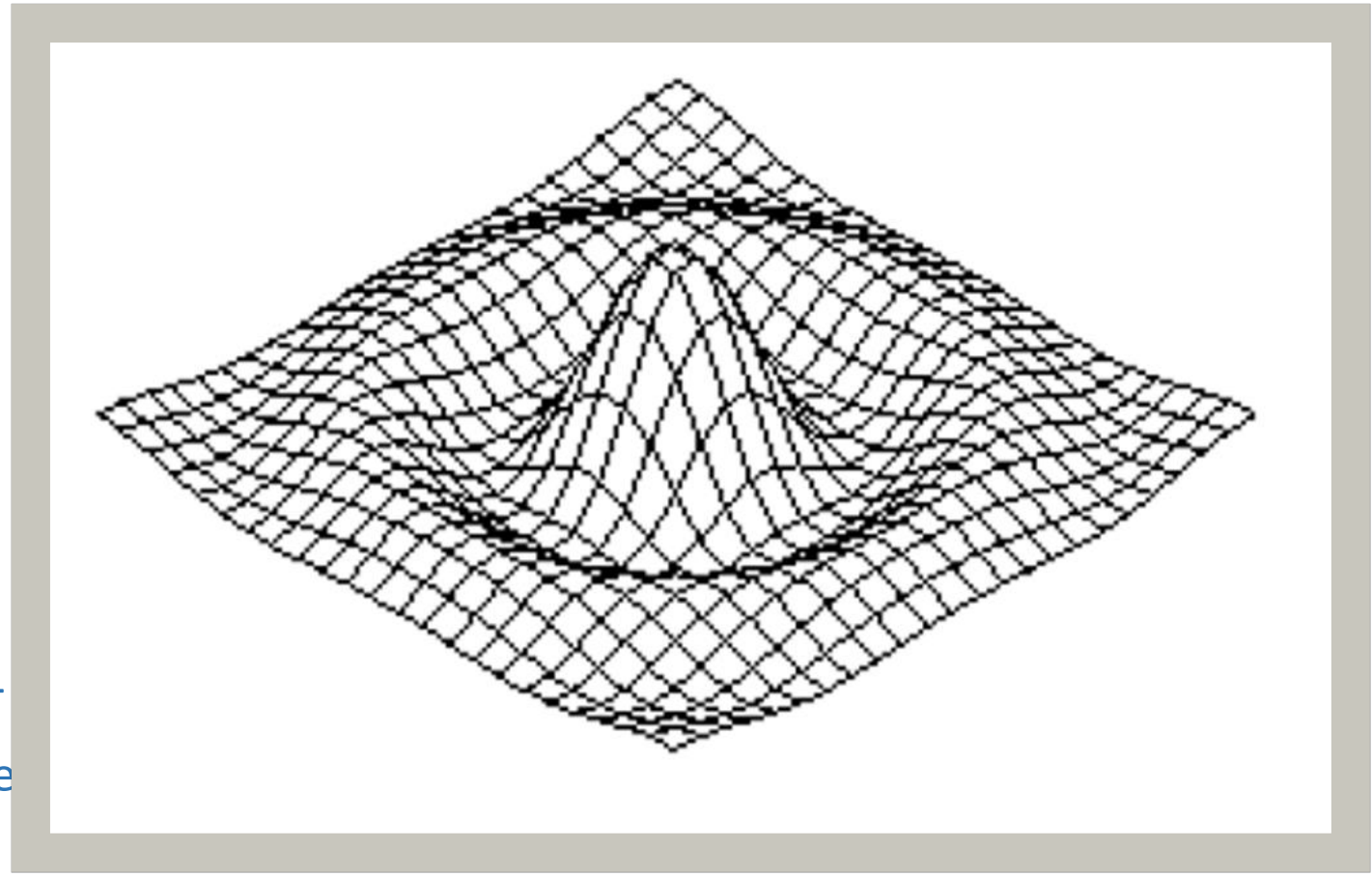
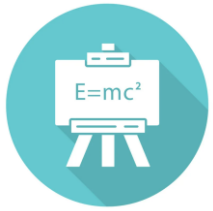


График функции Z=SIN(R)/R, где R = SQR (X*X + Y*Y)

Прогресс не остановить



- ***Вайб-кодинг***
- ***ЯП высокого уровня***
- ***Ассемблер***
- ***Машинный код***



Что будем изучать? (Теория)

- Технология АОМК
  - Компоненты и интерфейсы
  - Интерфейс IEcoUnknown, UGUID
  - Повторная применяемость, Обратный вызов, Фабрика
  - События и точки подключения, Маршалинг
- Архитектура ОС
  - Микроядро
  - Интерфейсная шина, Загрузчик
  - Планировщик
  - Объекты синхронизации
  - Менеджер памяти
  - Файловый менеджер, Форматы исполняемых файлов



Чему научимся? (Практика)

- Настройке среды разработки под различные платформы и архитектуры
- Разработке компонентов по АОМК - технологии:
  - для прикладных задач
  - для низкоуровневых, системных задач ОС
- Разработке решений с использованием компонентов
  - Добавление нового функционала в ПО Калькулятор
  - Создание микроядра ОС с разными алгоритмами планирования
- Программированию микроконтроллеров и систем на кристалле
  - Bare metal программирование по тех. спецификациям (data sheet)
  - Разработка драйверов аппаратной части



Как будут оцениваться полученные знания?

- Оценка по АОМК (ACOM) технологии
  - Разработка компонента согласно выбранному заданию.
- Оценка по построению ОС
  - Разработка компонента согласно выбранному заданию.



Где применить полученные знания и навыки?

- Полученные знания по АОМК (ACOM) технологии, помогут в изучении и понимании аналогичных компонентных технологиях таких как COM, DCOM, OLE, ActiveX, XPCOM, CORBA.
- Помогут в проектирование решений, где необходимо использовать язык описания интерфейсов IDL, таких как MIDL (Microsoft IDL), AIDL (Android IDL), IDL DCE & COM IDL (Open Group), CORBA IDL (OMG)
- Разработке прикладного ПО в различных направлениях IT индустрии на компонентном подходе и на различных платформах
- В проектирование и разработке специализированных ОС для встраиваемых систем (умные вещи, автомобили, робототехника, станки с ЧПУ и т.п.)



Курсовые и дипломные работы.

- Тема
  - Любая интересующая тема из IT (требуется согласование)
  - Тема на выбор из предложенного списка по данному курсу «Проектирование ОС».
  - Тема на выбор из предложенного списка по курсу «Проектирование компилятора».
- Требования
  - Использование технологии АОМК (ACOM).
  - Использование компонентов платформы Eco.Framework
  - Реализация на языке программирование Си (стандарт 1989)



Почему Си?



Популярность ЯП Си



Индекс TIOBE - индекс, оценивающий популярность ЯП

Programming Language	2026	2025	2024	2023	2018	2013	2008	2003	1998	1993	1988
Python	1	1	1	1	4	8	6	11	24	22	-
C	2	4	2	2	2	1	2	2	1	1	1
C++	3	2	3	3	3	4	3	3	2	2	4
Java	4	3	4	4	1	2	1	1	18	-	-
C#	5	5	5	5	5	5	8	9	-	-	-
JavaScript	6	6	6	6	8	9	9	8	21	-	-
VisualBasic	7	9	10	7	18	-	-	-	-	-	-
PHP	13	13	12	8	7	6	5	6	-	-	-
SQL	9	8	7	9	88	-	-	7	-	-	-
Assembly language	19	17	11	10	13	-	-	-	-	-	-

Язык Си, становился языком года в 2008, 2017, 2019 г

Ссылка: https://www.tiobe.com/tiobe-index/



Популярность ЯП Си

IEEE Advancing Technology for Humanity

Рейтинг ЯП от Института инженеров электротехники и электроники

Programming Language	2025	2024	2023	2022	2021	2020	2019	2018	2017	2016	2015	2014
Python	1	1	1	1	1	1	1	1	1	3	4	4
Java	2	2	2	5	2	2	2	3	3	2	1	1
C++	3	4	3	3	4	4	4	2	4	4	3	3
C	8	9	4	2	3	3	3	4	2	1	2	2
JavaScript	6	3	5	7	5	5	6	8	7	8	8	6
C#	5	7	6	4	6	23	7	5	5	6	5	5
SQL	4	6	7	6	15	13	23	23	23	24	11	10
Go	10	8	8	11	8	8	10	9	9	10	20	15
TypeScript	7	5	9	10	-	-	-	-	-	-	-	-
HTML	18	10	10	9	9	17	14	14	16	16	14	-

Язык Си, становился языком года в 2016 г.

Ссылка: https://spectrum.ieee.org/the-top-programming-languages-2024



Популярность ЯП Си

- Стандарт языка Си:
  - K&R C - 1978, Brian Kernighan and Dennis Ritchie
  - C89 (ANSI C) - ANSI X3.159-1989
  - C90 - ISO/IEC 9899:1990
  - C99 - ISO/IEC 9899:1999
  - C11 - ISO/IEC 9899:2011
  - C17 - ISO/IEC 9899:2018
  - C23 - ISO/IEC 9899:2024
  - C2Y - ISO/IEC 9899:(2029-2031)

ISO

международная организация по стандартизации

- MISRA C - стандарт разработки ПО на языке Си
  - First edition 1998 (127 правил).
  - Second edition 2004 (141 правило).
  - Third edition 2012 (143 правила),
    - updated 2016, 2020, 2023 (156 правил)

MISRA

Ассоциация надёжности программного обеспечения автомобильной промышленности



Популярность ЯП Си

- На языке Си написаны ЯП:
  - C++, C#, Objective-C, Python, PHP, Perl …



- На языке Си написаны ОС и драйвера:
  - Windows, Linux, Mac OS, Android …



- Язык Си применяется в областях, где требуется высокая надежность:
  - Космос, авиация, ядерная энергетика, встраиваемые системы …
- По данным крупнейшего каталога программ с открытым исходным кодом «Black Duck» более 45% этих программ написано на языке Си.

Примеры выполненных КР и ДР.

- ***ТЕМА ВКР: Алгоритмы распознавания на базе компьютерного зрения для устройств Интернета вещей***

В данной работе алгоритм разрабатывается для решения задачи распознавания лиц. Суть алгоритма в представлении изображения как суммы базисных компонент.

$$\Phi_i = \sum_{j=1}^{K} w_j u_j$$

Здесь Φi – это нормированное изображение выборки. Wi – это веса алгоритма. Ui – это собственные вектора.

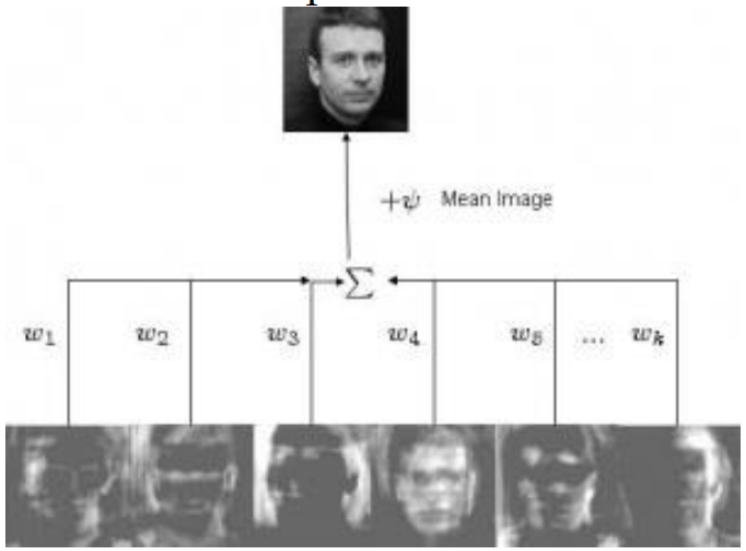


Рис.14.Разложение фотографии по собственным векторам.

ТЕМА ВКР: Алгоритмы распознавания на базе компьютерного зрения для устройств Интернета вещей

Архитектура компонент

Все разработанные компоненты имеют архитектуру, общую для всех компонент технологии АОМК. Компоненты имеют структуру - интерфейс, который хранит таблицу всех открытых для пользователя функций, но все данные и внутренние функции компонента хранятся в другой структуре – непосредственно компоненте, указатель на которую можно получить, приведя указатель интерфейса к указателю структуры компонента. Для того, чтобы иметь доступ к данным структуры, в каждую из функций компонент в качестве параметра передаётся указатель на структуру интерфейса.

Реализация компонента Eigenfaces

Алгоритм Eigenfaces был реализован в виде встраиваемой компоненты в рамках технологии АОМК на языке Си стандарта C89. Интерфейс компоненты состоит из следующих функций:

```
int16_t (*Train)(/* in */ struct IEcoEigenfaces1* me, uint8_t** bitmaps,
                 /* in */ uint32_t bitmapSize, int32_t numImages);

int16_t (*Save)(/* in */ struct IEcoEigenfaces1* me, int32_t* labels,
                /* in */char_t* filenameEigenVectorsBinary, char_t* filenameWeightsBinary,
                /* in */ char_t* filenameLabels);

int16_t (*Load)(/* in */ struct IEcoEigenfaces1* me, char_t* filenameEigenVectorsBinary,
                /* in */ char_t* filenameWeightsBinary, char_t* filenameLabels);

int16_t (*Predict)(/* in */ struct IEcoEigenfaces1* me, uint8_t* bitmap, uint32_t bitmapSize,
                   /* out */ int32_t* predictedLabel);
```

Рис.16. Интерфейсы реализованного алгоритма.

Разберём подробно каждую функцию.

1. Функция Train получает на вход массив битовых карт изображений обучающей выборки, размер каждой карты и их количество. Битовая карта представляет из себя пиксели изображения, записанные в массив, каждый пиксель представлен тремя целыми числами (RGB компоненты). Функция совершает следующие шаги:

Алгоритмы распознавания на базе компьютерного зрения для устройств Интернета вещей

Для визуализации алгоритма усреднённый вектор был выведен на экран.

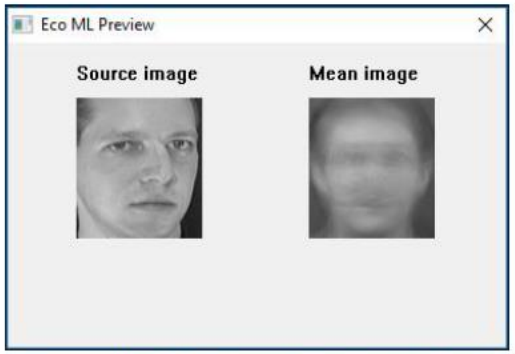


Рис.17. Усреднённое изображение.

Затем было выведено то же изображение, но после вычитания среднего вектора.

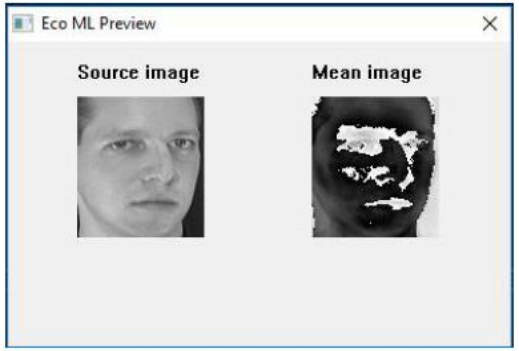


Рис.18. Изображение после вычитания среднего вектора.

2. Функция Save сохраняет полученные на этапе обучения параметры в файлы, пути к которым передаются через параметры функции.

ТЕМА КР: Создание встраиваемых ML компонент для IoT

3.1. Линейная регрессия.

В рамках данной работы был реализован алгоритм линейной регрессии, который позволяет эффективно решать задачу регрессии. Данный алгоритм является базовым и имеет множество преимуществ, а именно подходит для решения большого множества задач, является хорошо интерпретируемым, так как по весам конкретных признаков можно понять вклад каждого признака в итоговое предсказание. Из минусов данного алгоритма можно выделить его требовательность к данным, среди них не должно быть коррелирующих признаков и данные должны быть предобработаны.

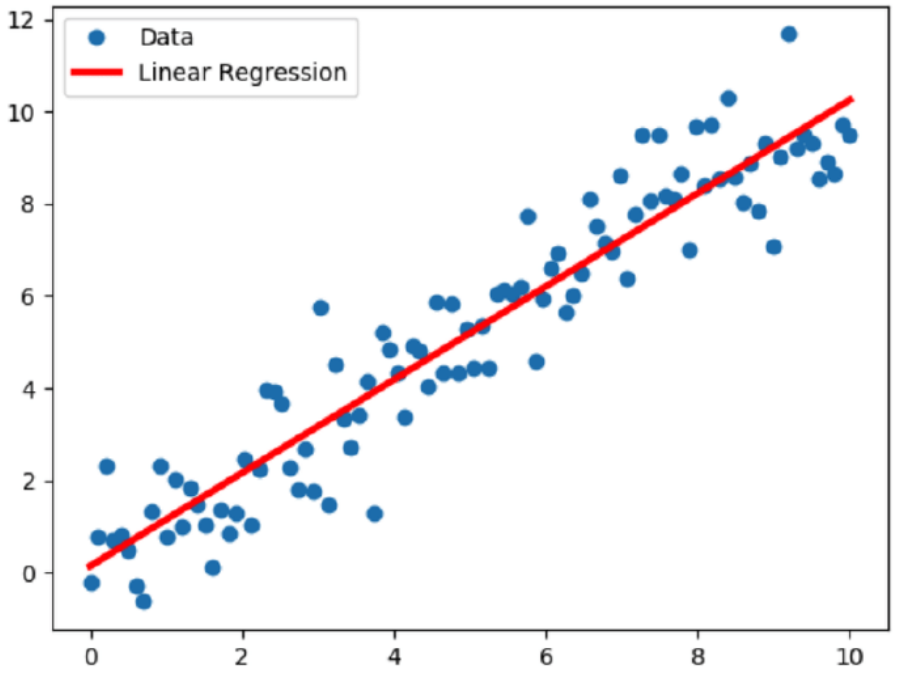


Рис.6. Пример предсказания линейной регрессии.

Создание встраиваемых ML компонент для IoT.

В таблице представлены метрики на тестовой и обучающей выборках.

Модель	MSE на обучающей выборке	MSE на тестовой выборке	Время обучения, с	Время тестирования, с
Линейная регрессия из данной работы (классический градиентный спуск)	0.44	0.51	0.029	0.001
Модель LinearRegression из библиотеки Sklearn (метод наименьших квадратов)	0.43	0.38	0.038	0.0025
Модель SGDRegressor из библиотеки Sklearn (стохастический градиентный спуск)	0.44	0.37	0.058	0.0008

Из таблицы можно сделать вывод, что качество модели на обучающих выборках не отличается, однако реализованная модель на тестовой выборке оказалась немного менее точная, чем модели из sklearn, что значит, что модель имеет небольшую склонность к переобучению. Время работы реализованной в курсовой работе модели оказалось несколько лучше на обучении и тестировании по сравнению с аналогами из sklearn, однако время работы метода Predict у модели SGDRegressor оказалось несколько меньше, чем у реализованной модели.

ТЕМА КР: Исследование коммуникационных технологий внутри автомобиля

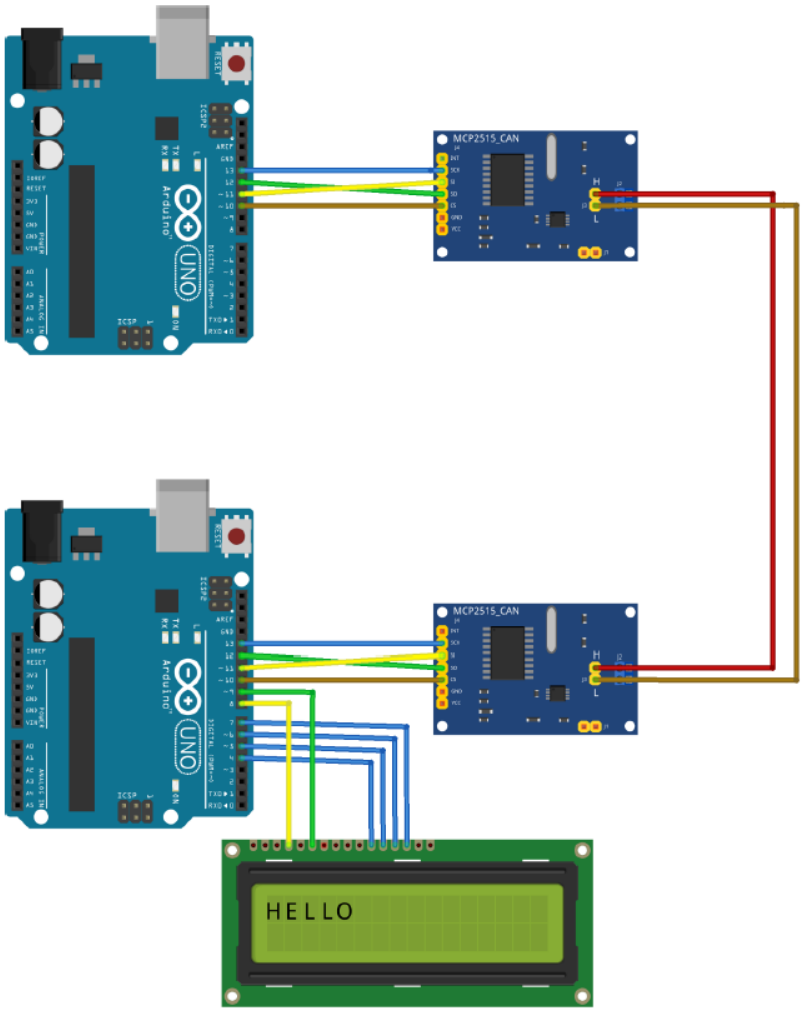


рис.12 Схема соединения устройств

Исследование коммуникационных технологий внутри автомобиля

5. Программная реализация

5.1 Передача данных по SPI

При реализации компонента для передачи данных по SPI были реализованы следующие функции:

```
int16_t (*Init)(/* in */ struct IEcoSPI1* me, /* in */ bool_t master);
uint8_t (*Transmit)(/* in */ struct IEcoSPI1* me, /* in */ uint8_t data);
uint8_t (*Receive)(/* in */ struct IEcoSPI1* me);
int16_t (*set_Mode)(/* in */ struct IEcoSPI1* me, /* in */ uint8_t mode);
```

рис.13 Функции, обеспечивающие передачу данных по SPI

Функция Init

Это функция инициализации SPI модуля. Внутри нее, если устройство настраивается в режиме Master, MISO настраивается как вход, MISO, SCK и SS как выход. Кроме того, устанавливается Mode0, частота тактового сигнала $F_{osc}/4$ по умолчанию, порядок передачи данных. Если устройство инициализируется как Slave, MOSI, SCK и SS инициализируются как входы, а MISO – выход.

```
int16_t CEcoSPI1_Init(/* in */ struct IEcoSPI1* me, /* in */ bool_t master) {
    CEcoSPI1* pCMe = (CEcoSPI1*)me;

    /* Проверка указателя */
    if (me == 0 ) {
        return -1;
    }
    /* Инициализация ведущего */
    if (master == 1) {
        /* Настройка MISO как вход, остальное как выход: */
        (*(volatile uint8_t *)(pCMe->m_PortDDR)) = (1<<pCMe->m_MOSI)|(1<<pCMe->m_SCK)|
(1<<pCMe->m_SS)|(0<<pCMe->m_MISO);
        (*(volatile uint8_t *)(pCMe->m_PortDR)) |= (1<<pCMe->m_SS);
        /* Разрешение SPI, старший бит вперед, мастер, SPI mode0, частота Fosc/4 */
        (*(volatile uint8_t *)(pCMe->m_SPCR)) = (1<<ECO_SPI_SPE)|(0<<ECO_SPI_DORD)|
(1<<ECO_SPI_MSTR)|(0<<ECO_SPI_CPOL)|(0<<ECO_SPI_CPHA)|(0<<ECO_SPI_SPR1)|
(0<<ECO_SPI_SPR0);
        (*(volatile uint8_t *)(pCMe->m_SPSR)) = ((*(volatile uint8_t *)(pCMe->m_SPSR)) &
~0x01) | (0 <<ECO_SPI_SPI2X);
    }
    /* Инициализация ведомого */
    else {
        (*(volatile uint8_t *)(pCMe->m_PortDDR)) = (0<<pCMe->m_MOSI)|(0<<pCMe->m_SCK)|
(0<<pCMe->m_SS)|(1<<pCMe->m_MISO);
        /* Разрешить работу SPI: */
        (*(volatile uint8_t *)(pCMe->m_SPCR)) = (1<<ECO_SPI_SPE)|(0<<ECO_SPI_MSTR);
    }
```

ТЕМА КР: Разработка и исследование библиотеки синтаксического анализа

5.5. Применение LR-анализа

Итак, рассмотрев различные варианты LR-анализа можно сделать следующие выводы:

1. LR-анализ, подобно LL-анализу позволяет анализировать строку за линейное время.
2. LR-анализ позволяет анализировать более широкий класс грамматик, нежели LL [19; 2]. Именно из-за этого в общем случае LR-анализ является более популярным (рис. 19).

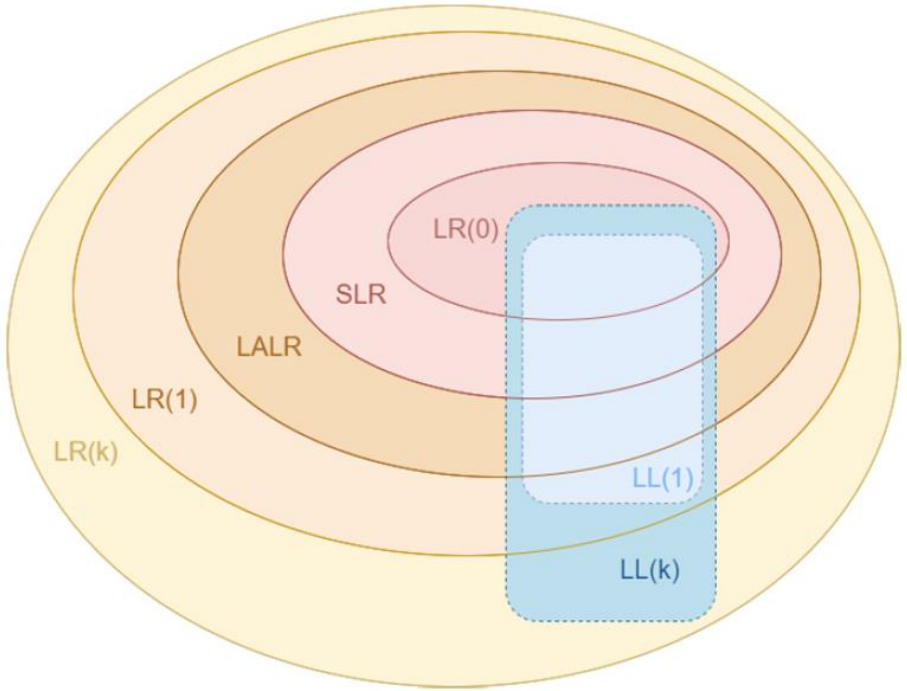


Рисунок 19. Взаимосвязь между классами грамматик

Разработка и исследование библиотеки синтаксического анализа

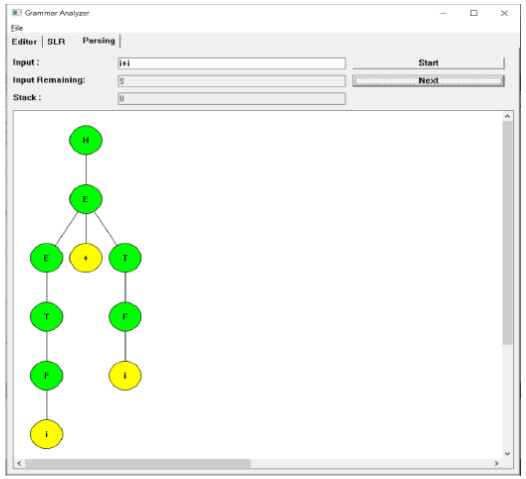


Рисунок 33. Пример SLR-грамматики: построение дерева разбора

Пример LR(1)-грамматики

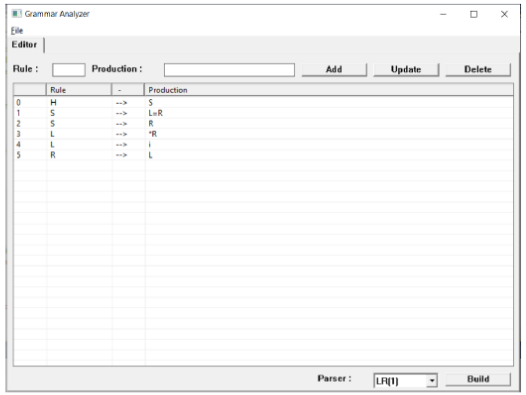


Рисунок 34. Пример LR(1)-грамматики: ввод грамматики

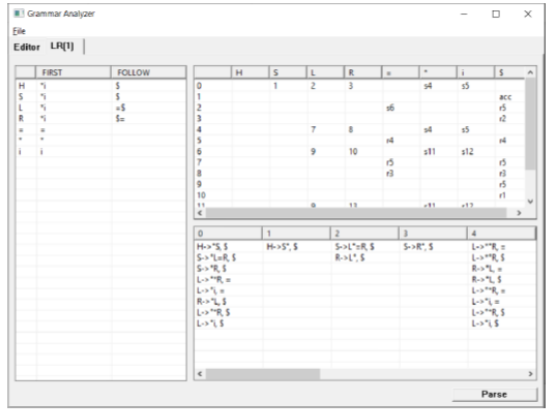


Рисунок 35. LR(1)-грамматики: построение множеств и таблиц

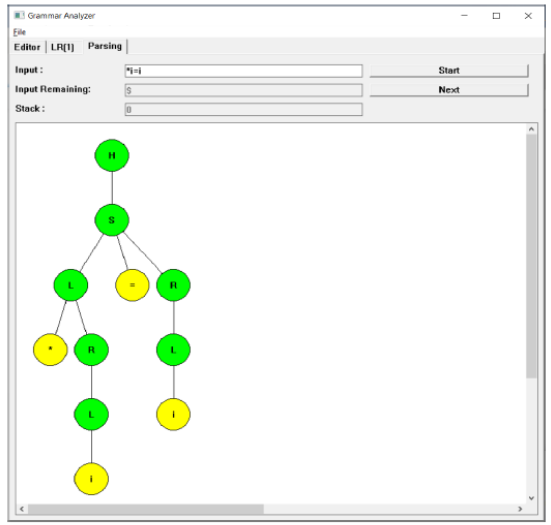


Рисунок 36. LR(1)-грамматики: построение дерева разбора

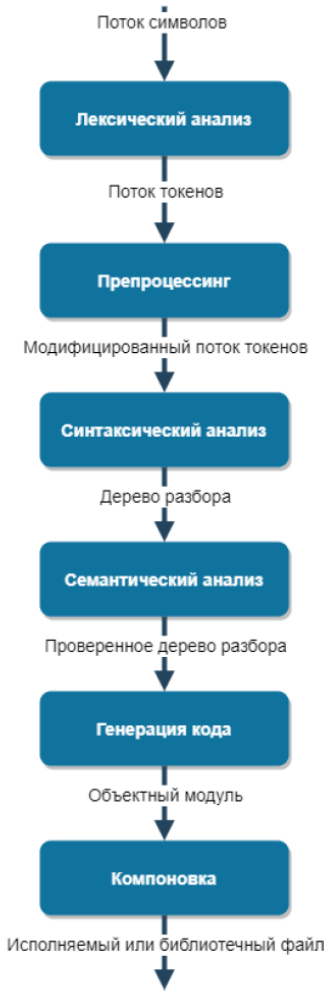
ТЕМА ДР: Разработка инструментов для создания компилятора

1.1. Этапы компиляции программы

Как уже было сказано выше, процесс компиляции представляет из себя некоторый конвейер. Этот конвейер получает на вход исходный код программы, а затем преобразует его форму, последовательно выполняя этапы обработки. В общем виде процесс для языка Си разделяется на следующие этапы:

Первым этапом является **лексический анализ**, который получает на вход поток символов исходной программы и разделяет их на токены (слова языка), определяя их тип и другие особенности, необходимые для дальнейшей обработки. В результате на выходе программы получается поток токенов.

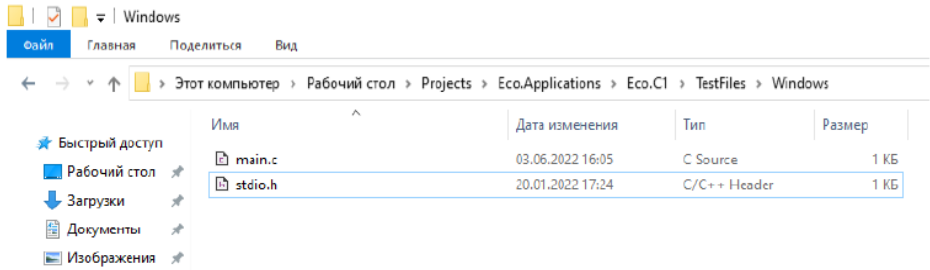
Вторым этапом компиляции является **препроцессинг**, или предварительный анализ токенов, полученный на предыдущем этапе. Здесь происходит анализ директив компилятора, подключение к потоку содержимого включаемых заголовочных файлов, пропуск некоторых разделительных токенов, которые не влияют на дальнейшие этапы, такие как пробелы, переводы



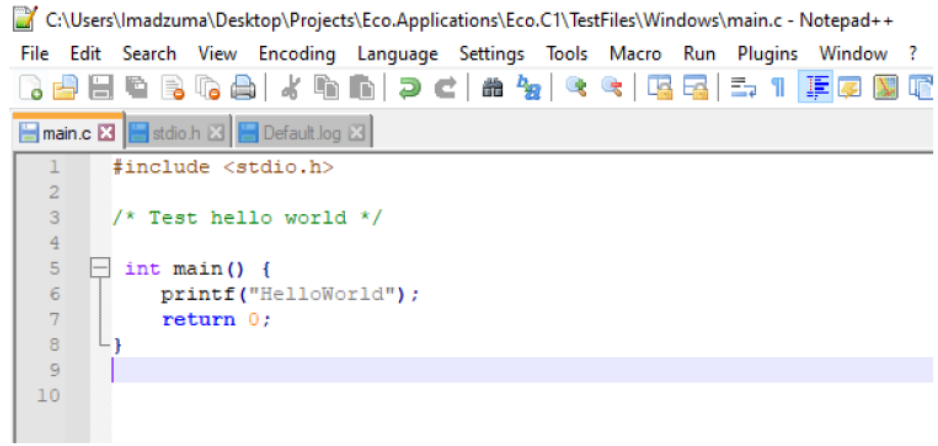
Разработка инструментов для создания компилятора

Приложение 2: пример работы компилятора

Пусть в папке с тестовыми файлами имеется 2 файла:



Один из них – main.c, файл с исходным кодом, содержащий популярную программу «Hello World»:



ТЕМА ВКР: Разработка компонента растрового графического формата JPEG по АОМК технологии для компьютерной графики

Кодирование и декодирование JPEG

Процесс проектирования реализации компонента по работе с JPEG можно разбить на две большие смысловые группы: декодирование и кодирование. Во время кодирования будет использоваться BitMap файл, потому что он содержить RGB представление картинки и заголовок, в котором есть вся необходимая информация, такая как размеры изображения, количество цветовых плоскостей, количество бит на пиксель и т.д. Декодирование же будет преобразовывать JPEG файл в Bitmap.



Кодирование

Для кодирования изображения в формат JPEG необходимо сначала прочитать заголовок BitMap файла, чтобы получить необходимую информацию о размере изображения и количестве бит, для кодирования пикселя.

Затем преобразовать RGB пиксели из Bitmap в формат YCbCr - другой формат записи цветов, который используется в JPEG. Для этого используются следующие формулы:

YCbCr:

$$Y = 0.299\ R + 0.587\ G + 0.114\ B$$
$$Cb = 0.1678\ R - 0.3313\ G + 0.5\ B$$
$$Cr = 0.5\ R - 0.4187\ G + 0.0813\ B$$

Разработка компонента растрового графического формата JPEG

"16 0". Это позволяет существенно уменьшить размер файла JPEG и ускорить передачу данных.



Код Хаффмана

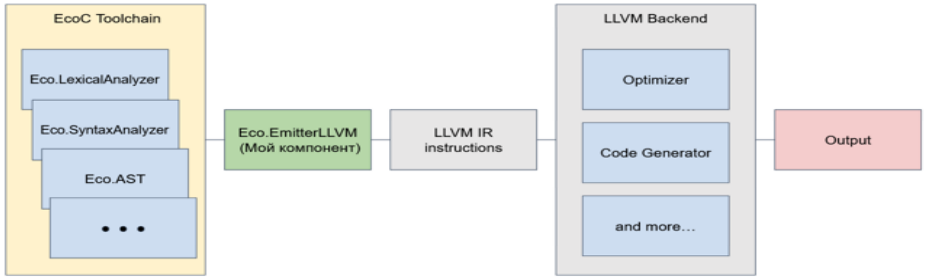
Кодирование Хаффмана - это метод сжатия данных, который использует переменную длину кодовой последовательности для представления различных символов в исходном наборе данных. В JPEG, каждый квантованный DCT-коэффициент представляется как целое число, которое может быть закодировано с использованием кодирования Хаффмана. Кодирование Хаффмана использует таблицу частот символов в исходном наборе данных для создания оптимальных кодовых последовательностей для каждого символа. Часто встречающиеся символы получают короткие коды, а редко встречающиеся символы получают длинные коды. Это позволяет сжимать данные, используя меньшее количество битов для часто встречающихся символов и большее количество битов для редко встречающихся символов. Кодирование Хаффмана помогает уменьшить размер файла JPEG и ускорить передачу данных.

ТЕМА ВКР: Разработка генератора инструкций LLVM IR для интеграции с набором инструментов Eco.Framework

5. Проектирование реализации

5.1. Общая архитектура

Общая архитектура системы представлена на диаграмме ниже:



Изображение 1. Диаграмма архитектуры.

Слева на диаграмме представлен фронтенд EcoC. Фронтендом в компиляторе обычно называют ту его часть, которая отвечает за обработку текстовых файлов с исходным кодом на различных языках программирования и создание внутреннего представления. Говоря о компонентно-ориентированном подходе, фронтенд будет представлять собой набор независимых компонентов, реализующих каждый свою задачу (препроцессинг, лексический анализ, синтаксический анализ и т.д.) и взаимодействующих между собой благодаря интерфейсам.

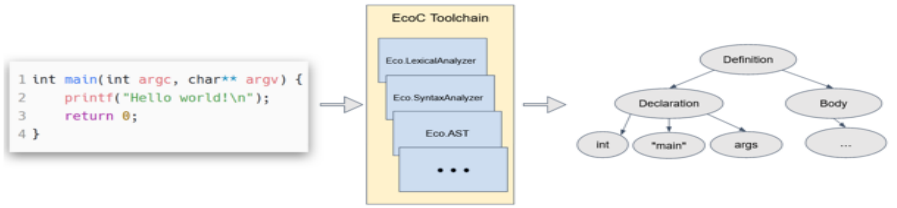
Справа на диаграмме представлен набор инструментов от LLVM, который необходимо интегрировать в EcoC в качестве бэкенда. Бэкендом здесь называется часть компилятора, ответственная за оптимизацию кода и кодогенерацию под выбранную аппаратную платформу.

Разрабатываемый компонент (отмечен зеленым цветом) является связующим звеном между этими двумя частями. Он использует интерфейсы EcoC для работы с внутренним представлением - AST (Абстрактное Синтаксическое Дерево) и интерфейсы LLVM для генерации LLVM IR инструкций, соответствующих каждой части исходной программы.

Результатом работы данной системы будет являться файл в бинарном или ассемблерном представлении под заданную аппаратную платформу (LLVM позволяет генерировать код не только под исходную архитектуру).

Разработка генератора инструкций LLVM IR

После того, как компилятор проводит лексический и синтаксический анализ исходного кода, он строит AST, который представляет собой удобную иерархическую структуру для дальнейшего анализа и оптимизации кода.



Изображение 2. Принцип работы AST.

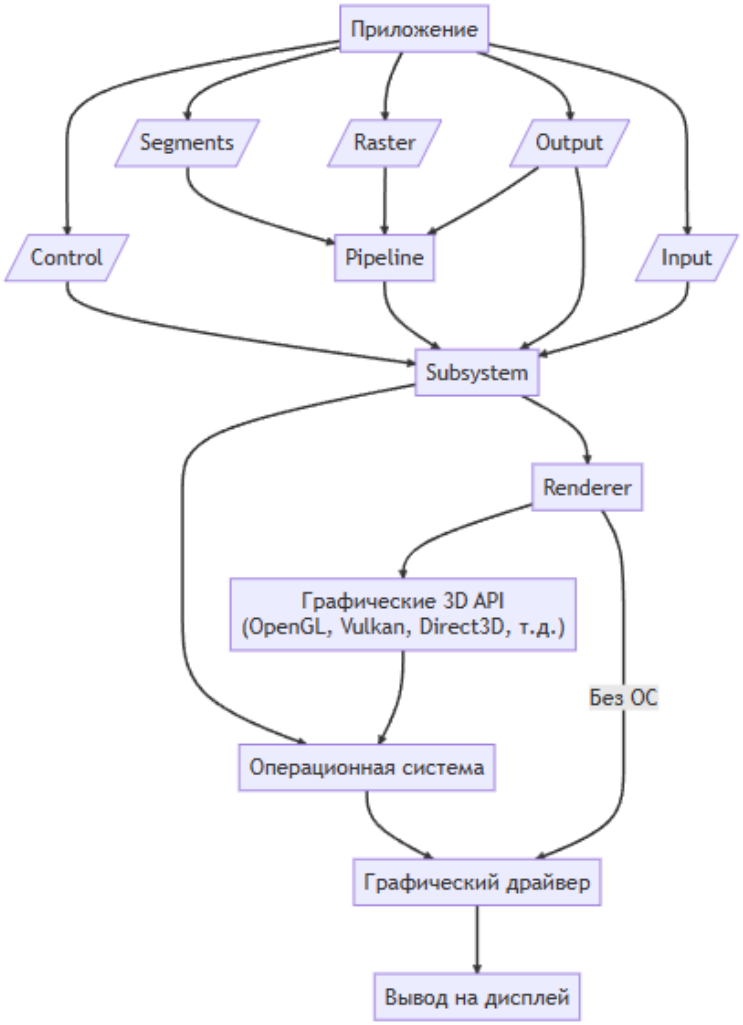
Абстрактные синтаксические деревья позволяют решать различные задачи, связанные с компиляцией программ, такие как оптимизация кода, генерация промежуточного кода и проверка синтаксиса программы. Благодаря своей удобной структуре, AST часто используются для автоматической генерации документации и облегчения отладки программ. В процессе компиляции, AST могут быть преобразованы и оптимизированы, например, для упрощения выражений или удаления неиспользуемых переменных. Кроме того, AST могут быть преобразованы в промежуточный код, такой как LLVM IR, для дальнейшей компиляции в машинный код.

LLVM IR.

LLVM IR (Intermediate Representation) - это промежуточное представление программы, которое используется в компиляторах. LLVM IR является абстрактным низкоуровневым языком, который позволяет описывать программу в виде последовательности инструкций, операций и данных. LLVM IR может быть использован для компиляции как для языков высокого уровня, так и для ассемблера.

Использование промежуточного представления довольно распространенный прием при создании компиляторов. Он позволяет упростить процесс компиляции и сделать его более эффективным. Компилятор может использовать IR для высокоуровневых оптимизаций (удаление неиспользуемого кода, свертка констант и т.д.) и генерации оптимального машинного кода для конкретного устройства или

ТЕМА ВКР: Создание графического фреймфорка для встраиваемых систем



Создание графического фреймфорка для встраиваемых систем

Описание отдельных компонентов EcoCGI1

Далее преведено подробное описание некоторых основных компонентов EcoCGI1.

EcoCGI1Output

Данный компонент выполняет функции вывода графических элементов на экран. С его помощью можно создавать новые объекты и задавать их атрибуты - цвет, стиль, позицию и т.д.

EcoCGI1Output предоставляет разработчику множество функций, например, функции работы с текстом:

```
// Добавление текста
int16_t Text(
        IEcoCGI1OutputPtr_t me,
        ECO_CGI_1_POINT_t* point,             // Позиция текста
        enum ECO_CGI_1_TEXT_FINAL_FLAG_e flag,// Флаг, определяющий
                                                                    // заканчивать текст или нет
        ECO_CGI_1_BDT_STRING_t string);       // Строка для отображения

// Добавление текста, ограниченного прямоугольником
int16_t RestrictedText(
        IEcoCGI1OutputPtr_t me,
    ECO_CGI_1_VDC_VALUE_t deltaWidth,     // Ширина ограниченного текста
        ECO_CGI_1_VDC_VALUE_t deltaHeight,    // Высота ограниченного текста
        ECO_CGI_1_POINT_t* point,             // Позиция текста
        enum ECO_CGI_1_TEXT_FINAL_FLAG_e flag,// Флаг, определяющий
                                                                    // заканчивать текст или нет
        ECO_CGI_1_BDT_STRING_t string);       // Строка для отображения

// Дополнение добовляемого текста
int16_t AppendText(
        IEcoCGI1OutputPtr_t me,
        enum ECO_CGI_1_TEXT_FINAL_FLAG_e flag,// Флаг, определяющий
                                                                    // заканчивать текст или нет
        ECO_CGI_1_BDT_STRING_t string)        // Строка для отображения
```

Функции работы со стандартными фигурами, например:

```
// Многоугольник, состоящий из списка точек
int16_t Polygon(
        IEcoCGI1OutputPtr_t me,
        ECO_CGI_1_POINT_LIST_t pointList);
```

Спасибо за внимание!